40-414 – Compiler Design, spring 2026, Sharif University of Technology

Programming Assignment II (Parser)

Released: Sunday, 17/03/1405

Due: Friday, 05/04/1405

1. Introduction

In programming assignment 1, you implemented a scanner. In this assignment you will write a **Predictive Top-Down parser** for C-minus, using the **LL(1) method** of lecture note 5. Using codes from text books, with a reference to the used book in your program is accepted. In this assignment, you can also use an online toolkit for computing the **First** and **Follow** sets of the non-terminals of the given C-minus grammar at **https://mikedevice.github.io/first-follow/**, plus a very useful piece of information called **Predict sets** for producing the Parsing Table. However, using codes from the internet and/or other groups/students in this course are **strictly forbidden** and may result in a **failure** grade in the course. Besides, even if you have not implemented the scanner in the previous assignment, you are not permitted to use scanners of other groups. In such a case, you need to implement both scanner and parser for this assignment. If you've announced and worked on previous programming assignment as a pair, you may continue to work on this assignment as **the same pair**, too.

2. Parser Specification

The parser that you implement in this assignment must have the following characteristics:

- The parsing algorithm **must be LL(1)** (i.e., based on the algorithm on page 6 of Lecture Note 5). **Please note that using any other parsing algorithm will not be acceptable and results a zero mark for this assignment.**
- Parsing is predictive, which means the parser never needs to backtrack.
- Parser works in parallel (i.e., pipeline) with the scanner and other forthcoming modules. In other words, your compiler must perform all tasks in a single pass.
- Parser calls **get_next_token** function every time it needs a token, until it receives the last token ('$'). Note that you need to modify your scanner in such way that it would return the token $, as the last token, when it reaches to the end-of-file of the input.
- Every time that parser calls function get_next_token, the current token is replaced by a new token returned by this function. In other words, there is only **one token** accessible to the parser at any stage of parsing.
- Parser recovers from the syntax errors using the **Panic Mode** method discussed on page 16 in Lecture Note 5 (and by using the **follow set** of each non-terminal as its **synchronizing set**).

In this assignment, similar to the previous assignment, the input file is a text file (i.e., named '**input.txt**'), which includes a C-minus program that is to be scanned and parsed. The parser's outputs include two text files, namely '**parse_tree.txt**' and '**syntax_errors.txt**', which respectively contain the parse tree and possible syntax errors of the input C-minus program. There is no need to print outputs of the scanner in this assignment.

3. C-minus LL(1) Grammar

The following grammar is a modified version of the C-minus grammar in [1], where the required conditions to have an **LL(1)** grammar (i.e., conditions on page 13 of Lecture Note 5) hold. Terminal symbols have a bold typeface in this grammar. **Please note that you must not change or simplify**

the following grammar in any ways.

1. Program → DeclarationList
2. DeclarationList → Declaration DeclarationList | EPSILON
3. Declaration → DeclarationInitial DeclarationPrime
4. DeclarationInitial → TypeSpecifier ID
5. DeclarationPrime → FunDeclarationPrime | VarDeclarationPrime
6. VarDeclarationPrime → ; | [NUM] VarDeclArrayPrime | = Expression ;
7. VarDeclArrayPrime → ; | = Expression ;
8. FunDeclarationPrime → (Params) CompoundStmt
9. TypeSpecifier → int | void
10. Params → int ID ParamPrime ParamList | void
11. ParamList → , Param ParamList | EPSILON
12. Param → DeclarationInitial ParamPrime
13. ParamPrime → [ ] | EPSILON
14. CompoundStmt → { DeclarationList StatementList }
15. StatementList → Statement StatementList | EPSILON
16. Statement → if (Expression) Statement ElseOpt | OtherStmt
17. ElseOpt → else Statement | EPSILON
18. OtherStmt → ID IdStatementPrime | SimpleExpressionZegond ; | ; |
    CompoundStmt | IterationStmt | ReturnStmt |
    BreakStmt | GotoStmt | SwitchStmt
19. IdStatementPrime → : Statement | B ;
20. BreakStmt → break ;
21. IterationStmt → while (Expression) Statement
22. ReturnStmt → return ReturnStmtPrime
23. ReturnStmtPrime → ; | Expression ;
24. Expression → SimpleExpressionZegond | ID B
25. B → = Expression | [Expression] H | SimpleExpressionPrime
26. H → = Expression | G D C
27. SimpleExpressionZegond → AdditiveExpressionZegond C
28. SimpleExpressionPrime → AdditiveExpressionPrime C
29. C → Relop AdditiveExpression | EPSILON
30. Relop → < | ==
31. AdditiveExpression → Term D
32. AdditiveExpressionPrime → TermPrime D
33. AdditiveExpressionZegond → TermZegond D
34. D → Addop Term D | EPSILON
35. Addop → + | -
36. Term → SignedFactor G
37. TermPrime → SignedFactorPrime G
38. TermZegond → SignedFactorZegond G
39. G → Mulop SignedFactor G | EPSILON
40. Mulop → * | /
41. SignedFactor → + Factor | - Factor | Factor
42. SignedFactorPrime → FactorPrime
43. SignedFactorZegond → + Factor | - Factor | FactorZegond
44. Factor → (Expression) | ID VarCallPrime | NUM
45. VarCallPrime → (Args) | VarPrime
46. VarPrime → [Expression] | EPSILON
47. FactorPrime → (Args) | EPSILON

48. FactorZegond → (Expression) | NUM
49. Args → ArgList | EPSILON
50. ArgList → Expression ArgListPrime
51. ArgListPrime → , Expression ArgListPrime | EPSILON
52. GotoStmt → goto ID ;
53. SwitchStmt → switch (Expression) { CaseList DefaultOpt }
54. CaseList → Case CaseList | EPSILON
55. Case → case Constant : StatementList
56. Constant → NUM
57. DefaultOpt → default : StatementList | EPSILON

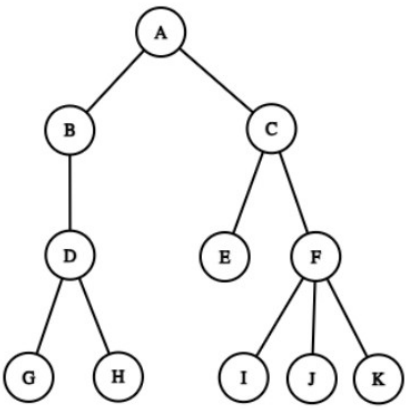
4. Parser Output

As it was mentioned above, your parser receives a text file named '**input.txt**' including a C-minus program and outputs the parse tree of the input program in a file named '**parse_tree.txt**'. Parser also produce a text file called '**syntax_errors.txt**', which includes error message regarding possible syntax errors. If there is no syntax error in the input program, a sentence '**There is no syntax error.**' should be written in '**syntax_errors.txt**'. Therefore, this output file should be created by the parser regardless of whether or not there exists any syntax error.

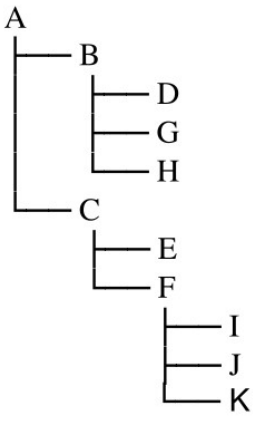
The parse tree inside '**parse_tree.txt**' should have the following format:

- Every line includes a node of the parse tree.
- The first line includes the root node, which is the start symbol of the grammar.
- In each line, the depth of the node in that line is shown by a number of tabs before the node's name.
- The successors of each node from left to right are respectively placed in the subsequent
- lines.

The following figures show an example a parse tree and the desired format of the output.



Sample Parse Tree



Sample Output

5. What to Turn In

Before submitting, please ensure you have done the following:

- It is your responsibility to ensure that the final version you submit does not have any debug print statements.
- You should submit a file named '**compiler.py**', or some python files together with the main file '**compiler.py**', which at this stage includes the Python code of your scanner and your LL(1) parser. Please write your **full name(s)** and **student number(s)**, and any reference that you may have used, as a comment at the beginning of 'compiler.py'.

- Your parser should be the main module of the compiler so that by calling the parser, the compilation process starts, and the parser invokes other modules such as scanner when it is needed.
- The responsibility of showing that you have understood the course topics is on you. Obtuse code will have a negative effect on your grade, so take the extra time to make your code readable.
- Your parser will be tested by running the command line '**python3 compiler.py**' in Ubuntu operating system using Python interpreter version 3.9. It is a default installation of the interpreter without any added libraries except for '**anytree**', which may be needed for creating the parse trees. No other additional Python's library function may be used for this or subsequent programming assignments. Please do make sure that your program is correctly compiled in the mentioned environment and by the given command before submitting your code. It is your responsibility to make sure that your code works properly using mentioned OS and Python interpreter.
- Submitted codes will be tested and graded using several different test cases (i.e., several 'input.txt' files) with and without syntax errors. Your program should read '**input.txt**' from the same working directory as your programs are placed. Please note that in the case of getting a compile or run-time error for a test case, a grade of zero will be assigned to your parser for that test case (Make sure your code does not return a non-zero exit code in case of success, so that it is not mistaken with an exception or error). Similarly, if the parser cannot produce the expected outputs (i.e., '**parse_tree.txt**' and '**syntax_errors.txt**') for a test case, a grade of zero will be assigned to it for that test case. Therefore, it is recommended that you test your scanner on several different random test cases before submitting your code.
- In a couple of days, you will also receive **10 input-output sample** files. Your code will be judged by **20 input-output samples**, including the mentioned 10 samples.
- Your parser will be evaluated by the Quera's Judge System (QJS).
- The decision about whether the scanner and parser functions to be included in the '**compiler.py**' or a as separate files such as, say '**scanner.py**' and '**parser.py**', is yours. However, all the required files should be reside in the same directory as '**compiler.py**'. In other words, We will place all your submitted files in the same plain directory including a test case and execute the '**python3 compiler.py**' command. You should upload your program files ('**compiler.py**' and any other text files that your programs may need) to the course page in Quera before the deadline.
- This project has no late-submission allowances. Deadlines are hard, and no unofficial delay is accepted. Extensions, if necessary, will be officially announced.
- Each group may design and submit up to three additional test cases. For each test case

that is correct and valid, the group will receive 0.1 bonus point added to the course grade. Invalid or incorrect tests receive no bonus.

- Each test case must include all of the following files:

    - input.txt
    - parse_tree.txt
    - syntax_errors.txt

    The format must be exactly identical to the sample test cases provided to you. Submit your test cases by emailing them to farzam.kooragh01@sharif.edu. The attachment must be named: **testcases_phase2_<group_number>.zip**. The email subject must be: **testcases - phase2 - group<group_number>**

- It is worth mentioning again that, as it was mentioned in phase 1, all the subsequent programming assignments will be dependent on the result of the previous one. Therefore, it is very important to implement the parser at this stage. Otherwise, one has to do this assignment later (without any credit) for the sake of subsequent assignments!